

DATA471 Assignment 4

Loren Whithair (Student ID: 300 434 521)

2026-09-16

Exercise 1

Regarding the computation of order statistics

Hyndman and Fan (1998) proposed 9 ways to compute the quantile over a set of sample data.

Generally speaking, to obtain the quantile p for ordered data x of size N , the function will look at $x[\text{floor}(p*N)]$ and $x[\text{floor}(p*N) + 1]$, and use one of two general methods to compute the quantile:

1. a discrete method that picks a value between these two ordered values in x set by another constant
2. a continuous interpolation method that “smooths” the quantile values when $x[\text{floor}(p*N)] <> x[\text{floor}(p*N) + 1]$

Extreme values of p (very close to 0 or 1) are handled explicitly to be $x[1]$ and $x[N]$ respectively.

Wicklin shows that for $n \geq 100$, the difference between the methods are minimal. Differences are noticeable for smaller datasets however, and so in these scenarios it’s important to report which one you used.

My version of R uses `type=7`, which is one of the continuous interpolation methods.

I would use the default unless there was strong reasoning for otherwise (e.g. small dataset where a discrete method is more appropriate given the context). Using the default is common practice and therefore makes it quicker simpler for other statisticians to understand outputs.

First-order bootstrap bias corrected estimator

First, we create the function that allows us to compute our original estimator.

```
lambda_iqr <- function(data){  
  return (log(3) / IQR(data))  
}  
set.seed(12345)  
# an example:  
x <- rexp(11, rate=1)  
print(lambda_iqr(x))
```

```
## [1] 1.415871
```

We then create a function that uses bootstrapping to estimated the expected value.

```
bootstrap_lambda <- function(data){
  boot_output <- rep(0, 200)
  for(1 in 1:200){
    boot_output[1] <- lambda_iqr(sample(data, size=length(data), replace=TRUE))
  }
  return(boot_output)
}
print(mean(bootstrap_lambda(x)))
```

```
## [1] 2.445498
```

Lastly, create the function that returns the bootstrap bias corrected estimator for λ_{IQR} :

```
lambda_tilda <- function(data){
  (2*lambda_iqr(data)) - mean(bootstrap_lambda(x))
}
```

Checking it works:

```
set.seed(1234567)
x <- rexp(100, rate=1)
paste0("Lambda IQR: ", lambda_iqr(x))
```

```
## [1] "Lambda IQR: 1.2043400638416"
```

```
paste0("Lambda Tilda: ", lambda_tilda(x))
```

```
## [1] "Lambda Tilda: 1.19813675838823"
```

Note that the inter-quartile range for a bootstrapped sample can be zero. This is more likely to occur for smaller x and larger B (bootstrapping iterations). This can make our parameter estimator infinite (divide by zero issue).

Monte Carlo Experiment

Firstly, we need to instantiate the final estimator function, based on the maximum likelihood:

```
lambda_ml <- function(data){
  return(mean(data))
}
# checking it works:
lambda_ml(rexp(10, rate=1))
```

```
## [1] 0.6396039
```

Next, we perform the Monte Carlo experiment.

Note that the number of repetitions chosen for each scenario is 5000. The more times we repeat the experiment per scenario, the closer we will converge measuring the true bias and variance. 5000 is considered sufficient for our analysis.

```

set.seed(1234567)

reps <- 5000 # the number of replications per lambda, n scenario

results <- data.frame()

for (lam in c(.1,1,10)){ #all lambda (true rate of the exponential) we are testing
  for (n in c(5,30,100)){# all sample sizes we are testing
    theta.ml <- rep(NA, reps)
    theta.iqr <- rep(NA, reps)
    theta.iqr_tilda <- rep(NA, reps)
    for (r in 1:reps){
      x <- rexp(n, rate=lam)
      theta.ml[r] <- lambda_ml(x)
      theta.iqr[r] <- lambda_iqr(x)
      theta.iqr_tilda[r] <- lambda_tilda(x) # lambda_iqr adjusted for bias with bootstrapping
    }

    theta.iqr_tilda[abs(theta.iqr_tilda)==Inf] <- NA
    inf_iqr_tilda_reps <- sum(is.na(theta.iqr_tilda))

    # calculate the bias of each theta, B
    # - uses law of large numbers (LLM) to approximate expected value as mean of multiple samples
    # - lam is the true statistics that we are trying to estimate
    bias.ml <- mean(theta.ml) - lam
    bias.iqr <- mean(theta.iqr) - lam
    bias.iqr_tilda <- mean(theta.iqr_tilda, na.rm=TRUE) - lam

    # calculate the variance of each theta
    var.ml <- var(theta.ml)
    var.iqr <- var(theta.iqr)
    var.iqr_tilda <- var(theta.iqr_tilda, na.rm=TRUE)

    # calculate the MSE
    mse.ml <- bias.ml^2 + var.ml
    mse.iqr <- bias.iqr^2 + var.iqr
    mse.iqr_tilda <- bias.iqr_tilda^2 + var.iqr_tilda

    # store the values
    results <- rbind(results, data.frame(
      lambda = lam,
      n = n,
      mse_ml = mse.ml,
      mse_iqr = mse.iqr,
      mse_iqr_tilda = mse.iqr_tilda,
      # how many of the IQR tilda values returned infinity, and therefore couldn't be used?
      prop_inf_reps = inf_iqr_tilda_reps/reps
    ))
  }
}

```

```

results_rounded <- results %>% mutate(
  mse_ml = round(mse_ml, 4),
  mse_iqr = round(mse_iqr, 4),
  mse_iqr_tilda = round(mse_iqr_tilda, 4)
)

colnames(results_rounded) <- c("Lambda",
                              "n",
                              "MSE (ML)",
                              "MSE (IQR)",
                              "MSE (IQR, bias-corrected)",
                              "Proportion of Infinite reps (IQR, bias-corrected)")

results_rounded

```

Results of the Monte Carlo

##	Lambda	n	MSE (ML)	MSE (IQR)	MSE (IQR, bias-corrected)
## 1	0.1	5	118.2473	0.4214	NaN
## 2	0.1	30	100.6130	0.0011	0.0013
## 3	0.1	100	99.0999	0.0003	0.0003
## 4	1.0	5	0.2022	43.6086	NaN
## 5	1.0	30	0.0331	0.1200	0.1334
## 6	1.0	100	0.0099	0.0259	0.0293
## 7	10.0	5	98.0167	2374.1343	NaN
## 8	10.0	30	98.0162	12.0191	13.2429
## 9	10.0	100	98.0057	2.5500	2.9271
##	Proportion of Infinite reps (IQR, bias-corrected)				
## 1					1
## 2					0
## 3					0
## 4					1
## 5					0
## 6					0
## 7					1
## 8					0
## 9					0

Table 1: The Mean Squared Error (MSE) for each estimator, for each scenario of (Lambda, n)

Referring to Table 1 we notice:

- For $n=5$, our bootstrap bias-corrected IQR estimator could not be computed for any Lambda. The function takes 200 bootstrapped samples from x . For x of smaller dimensionality, the odds of sampling the same value every time are increased. For these samples, the IQR equals zero, our lambda_IQR estimator is infinite / undefined, and our bias-corrected estimator equals negative infinity.
- For all three estimators, for all three Lambda, smaller sample sizes (smaller n) make it harder to estimate Lambda
- Our IQR based estimator is a clear improvement on the ML estimator, for larger n . When a sample size is smaller, our sample IQR is less likely to represent the true population's IQR, which may be causing this inaccuracy
- As to be expected, our bias-corrected IQR estimator is generally more accurate than our original IQR estimator, with the caveat that it is very difficult to compute for smaller sample sizes due to the bootstrapping method used.

Exercise 2

```
data(ausbeer)
```

1. Frequency

The frequency of the series is quarterly (also expressed as 4 observations per period)

This can be found with the `frequency` function:

```
frequency(ausbeer)
```

```
## [1] 4
```

Or by directly observing the number of columns per row:

```
head(ausbeer, n=20L)
```

```
##      Qtr1 Qtr2 Qtr3 Qtr4
## 1956  284  213  227  308
## 1957  262  228  236  320
## 1958  272  233  237  313
## 1959  261  227  250  314
## 1960  286  227  260  311
```

2. Overall behaviour

```
autoplot(ausbeer) +
  ggtitle("Quarterly Beer Production in Australia") +
  xlab("Quarter") +
  ylab("Beer produced in Megalitres (ML)")
```

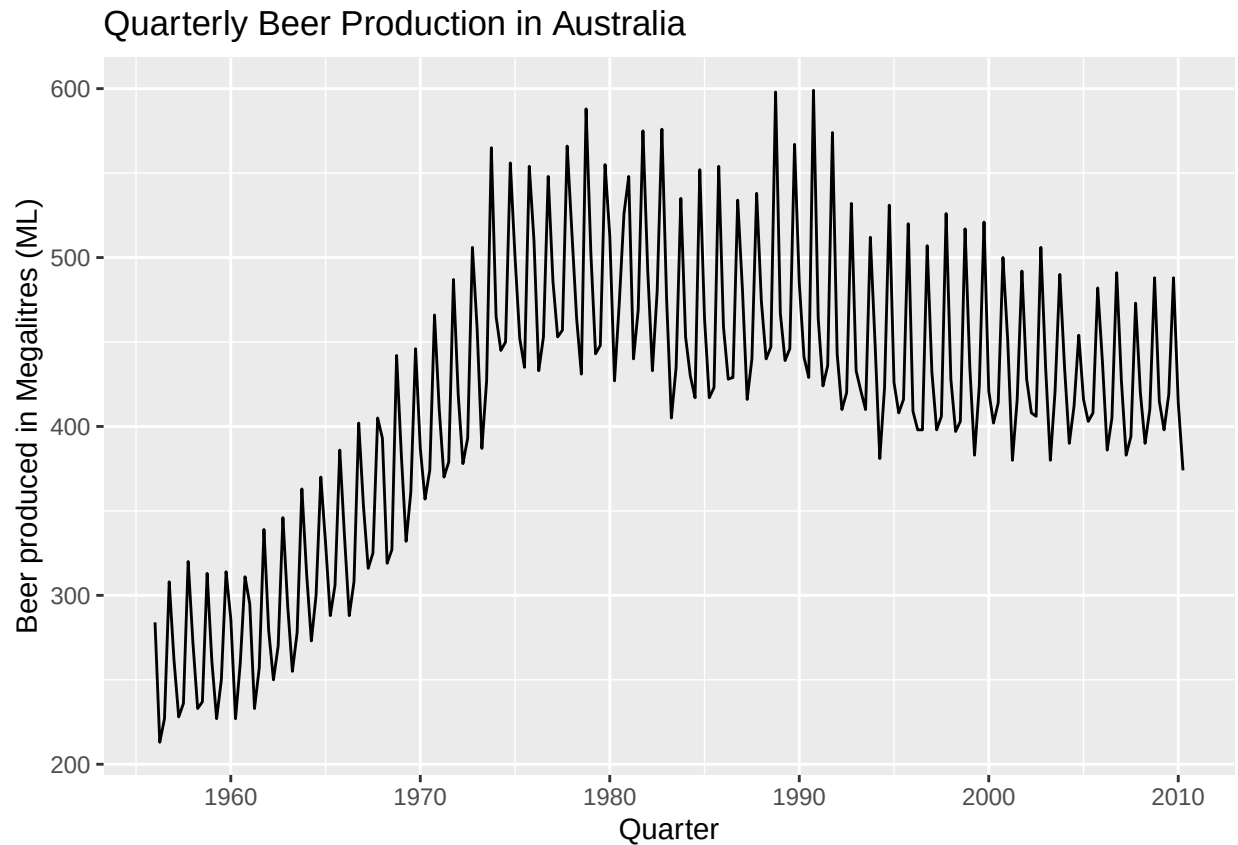


Figure 1: Time Series plot of quarterly beer production in MegaLitres

We see a long term trend in Figure 1: overall production increases dramatically from 1956 to approximately 1975, after which we see a much slower decline in production. Production may become steady around 2005, but without further data it is hard to tell if the downward trend has ended.

3. Seasonality

```
window(ausbeer, end=1970) %>%
  ggseasonplot(year.labels=FALSE) +
  ggtitle("Quarterly production of beer by season") +
  ylab("Beer produced in Megalitres (ML)")
```

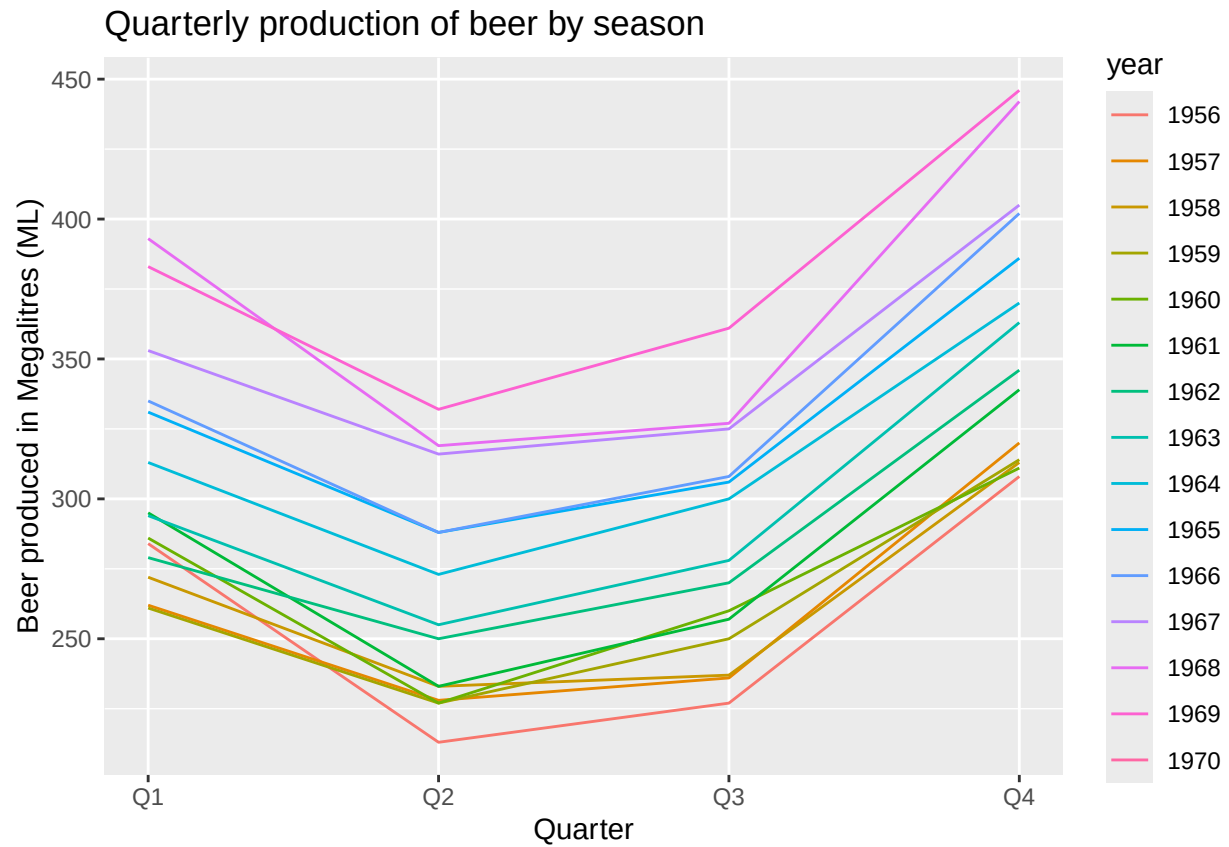


Figure 2.1: Season plot of quarterly beer production for years 1956 to 1970

```
window(ausbeer, start=1971, end=1995) %>%
  ggseasonplot(year.labels=FALSE) +
  ggtitle("Quarterly production of beer by season") +
  ylab("Beer produced in Megalitres (ML)")
```

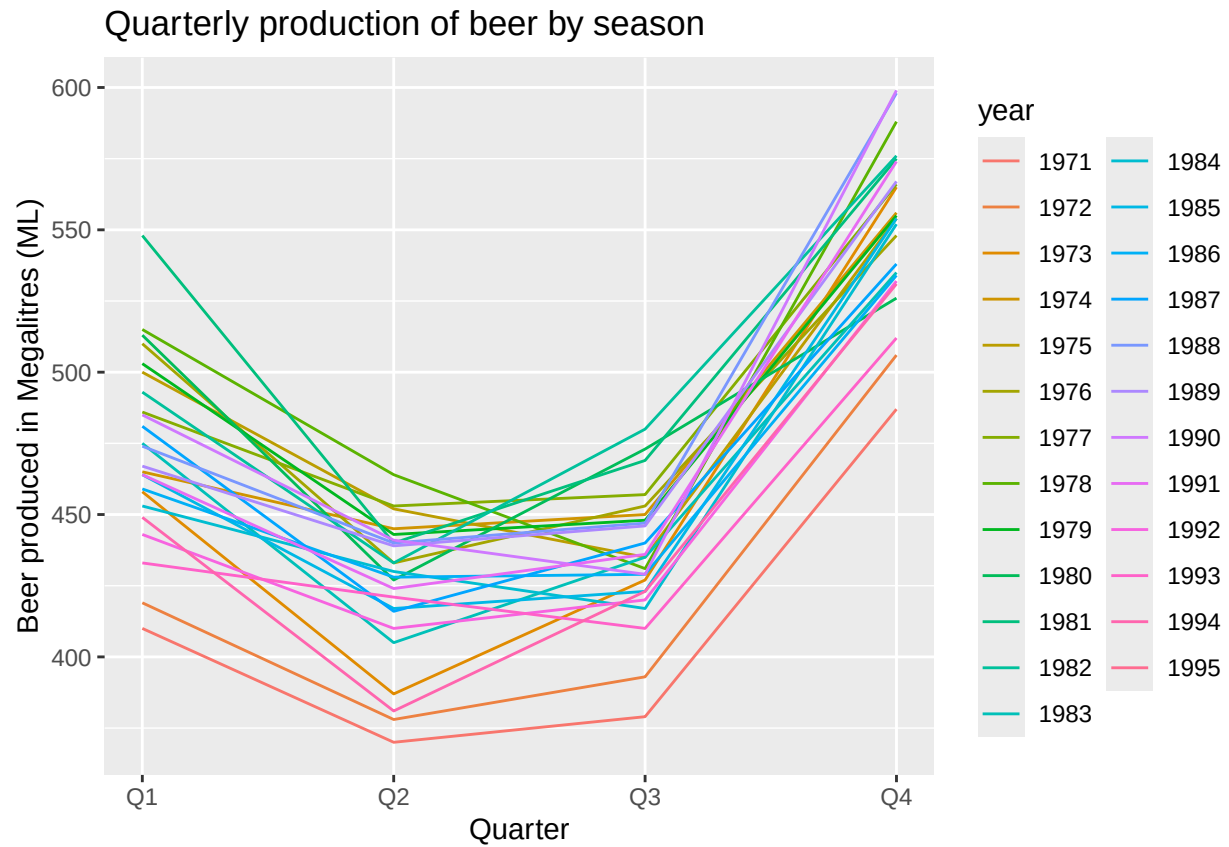


Figure 2.2: Season plot of quarterly beer production for years 1971 to 1995

```
window(ausbeer, start=1996, end=2010) %>%
  ggseasonplot(year.labels=FALSE) +
  ggtitle("Quarterly production of beer by season") +
  ylab("Beer produced in Megalitres (ML)")
```

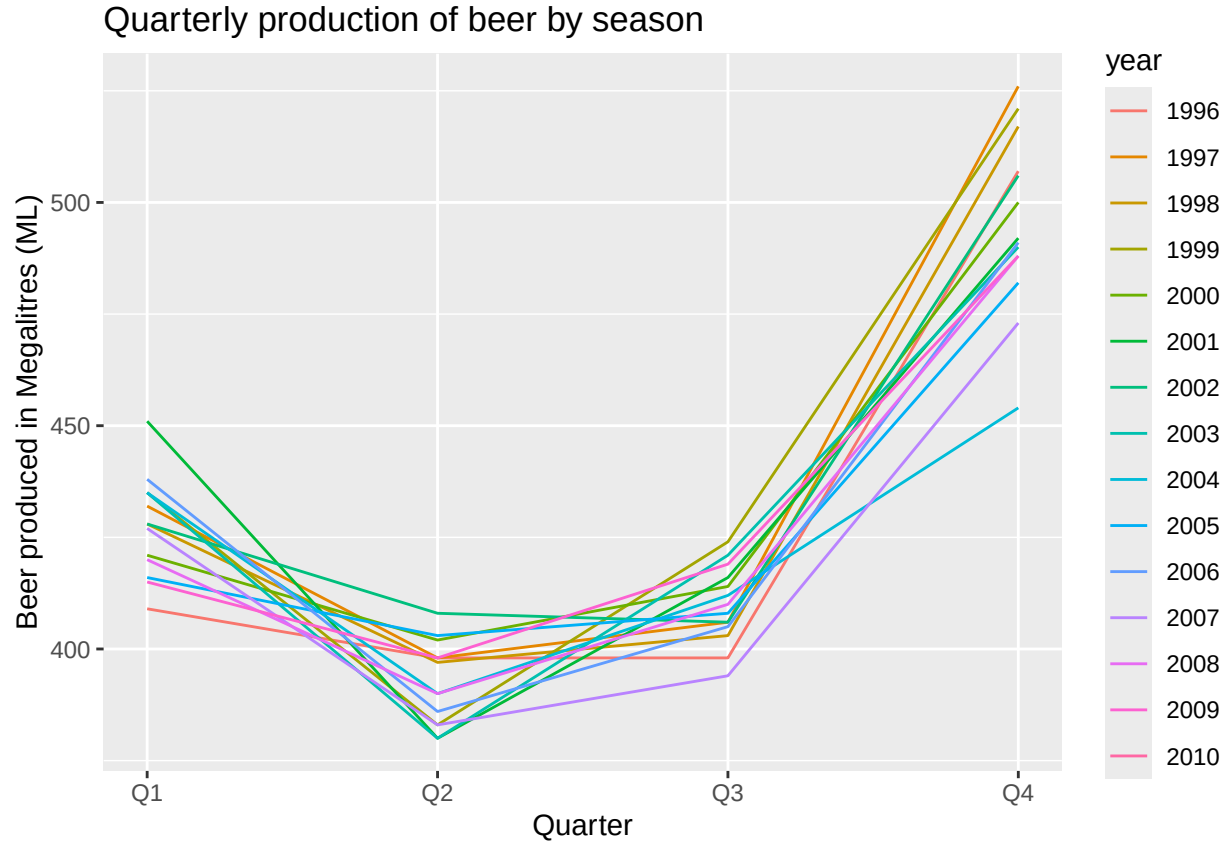


Figure 2.3: Season plot of quarterly beer production for years 1996 to 2010

Seasonality has been displayed on separate graphs for visibility of the two parts of the trend.

Figures 2.1 and 2.2 show clear seasonality: beer production is at it's highest for the year in Q4, closely followed by Q1. There is a drop in Q2 and Q3.

This may be due to more beer selling when the weather is warmer (Q4 is spring to early summer, Q1 is summer to early Autumn).

4. Variability

We see variance changes over time in Figure 1. We see an increase in variance around the mean per season after 1970 compared to before. This variance seems to slowly decrease along with the slow downward trend leading up to 2010.

The difference in variance is not dramatic, but if a model were to be fit it may be appropriate to explore transformations, if the original model does not perform well.

5. Unusual observations

Notice in Figure 1 that the trend for the lowest producing quarter (the lowest point in each curve) suddenly jumps up around 1974. Before this the upwards trend was very consistent.

While 1975 to 1990 can be considered more “volatile” than before and after this period, there are two comparable spikes in beer production around 1989 and 1991. Similarly, in 2005, much less beer is produced in the busiest quarter compared to other years after 2000. These stand out as they break our more consistent patterns.

6. Seasonal plots

See Figures 2.1, 2.2, and 2.3.

These plots makes the trend more apparent: for example, we can see in Figure 2.1 that later years exhibit more beer production across all quarters.

Additionally, it becomes easier to compare seasonality between years: for example, in Figure 2.3 we can see that Q2 frequently produces less beer than Q3, but that for some years such as the difference is negligible.

7. Sub-series plots

```
ggsubseriesplot(ausbeer) +  
  ggtitle("Seasonal subseries plot: beer production by quarter") +  
  ylab("Beer produced in Megalitres (ML)")
```

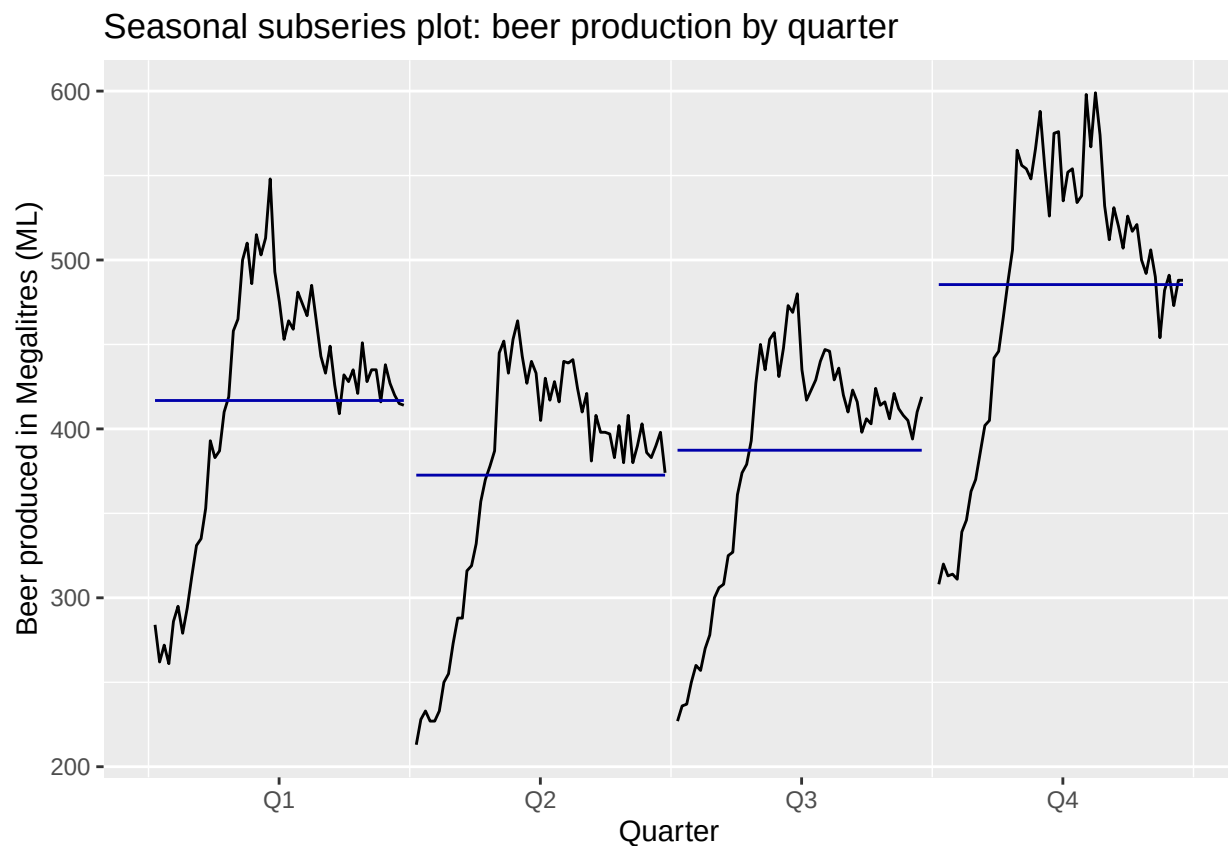


Figure 3: Sub-series plot of beer production within each quarter

The means for each quarter are shown by the blue bars in Figure 3.

Q2 on average has the lowest beer production, while Q4 has the highest on average.

All four sub-plots exhibit comparable shapes, indicating somewhat consistent seasonality. It is worth noting that the volatility and sudden spike in the overall trend is more apparent in Q1 and Q4 than in Q2 and Q3 (taller spikes can be seen in the Q1 and Q2 sub-plots). Given these are the highest producing quarters, it makes sense that in a more volatile state they would exhibit more variability.

8. Preparation for modelling

I would likely de-trend the data before training a model. Many modelling methods perform better under stationarity, which requires the mean, variance, and correlation with the past to be consistent. Figure 4 shows the effect of the trend: the mean shows a lot of variability.

As discussed in part (4), we see slight changes in the variability over time. A transformation may be appropriate, but it would be reasonable to try a model without a transformation first to see its performance.

```
df <- data.frame(  
  year=floor(time(ausbeer)),  
  beer=as.numeric(ausbeer)  
)  
  
df %>%  
  group_by(year) %>%  
  summarise(avg_production = mean(beer, na.rm=TRUE)) %>%  
  ggplot() +  
    geom_line(aes(x=year, y=avg_production)) +  
    ggtitle("Average quarterly beer production per year") +  
    ylab("Mean beer production across all quarters in the year (ML)") +  
    xlab("Year")
```



Figure 4: Mean annual beer production across quarters